Reproducing and Extending CLAP for Zero-Shot Audio Classification

Project Report

Shuvam Chakraborty (2024MCS2004)
Arfin Khan (2024MCS2001)
Bhuvnesh Kumar (2023MCS2011)

April 22, 2026

COL868 Special Topics in DBMS

Code Repository: https://github.com/arfinzz/CLAP-Special-Topic-DBMS

Abstract

This project studies *zero-shot audio classification* using CLAP (Contrastive Language-Audio Pretraining). The main task is to classify an input audio clip without task-specific supervised training by comparing its audio embedding against text embeddings generated from class prompts. Our work is not just a benchmark rerun: it is a reproduction-plus-extension study with a fully verified evaluation pipeline. We rebuilt the benchmark workflow, stabilized the environment in WSL2, ran the complete benchmark suite over `ESC-50`, `UrbanSound8K`, `GTZAN`, and `FSDD`, and produced reproducibility artifacts including dataset verification, checkpoint manifests, run manifests, and acceptance checks. The final run uses the official LAION CLAP implementation together with the stronger `630k-audioset-best.pt` checkpoint, so the most accurate description of the project is an *improved-checkpoint reproduction with extensions*. The main empirical findings are that CLAP transfers very well to environmental sound recognition, remains reasonably strong on urban sound and music genre classification, fails clearly on spoken-digit recognition, and does not obtain reliable gains from prompt ensembling.

1 Introduction and Motivation

The original CLAP idea is compelling because it extends the CLIP-style contrastive paradigm from image-text learning to audio-text learning. Instead of training a dedicated classifier for each downstream dataset, CLAP learns a shared embedding space in which audio clips and natural-language descriptions can be compared directly. This creates the possibility of *zero-shot transfer*: predicting classes by prompt matching rather than supervised retraining.

Our project was designed as a reproduction-plus-extension study rather than a single-number rerun. We wanted to answer the following questions:

- How well does CLAP transfer across environmental sound, urban sound, music genre, and speech-like tasks?
- Can the published benchmark-style setup be rebuilt in a verifiable environment?
- How sensitive are zero-shot results to prompt wording?

- Does prompt ensembling, inspired by CLIP-style prompt ensembling in vision, improve robustness?
- What happens when CLAP is tested on a narrow speech classification task rather than only sound and music tasks?

The project therefore had two main goals:

- reproduce the paper-style zero-shot benchmark on `ESC-50`, `UrbanSound8K`, and `GTZAN`;
- extend the study with a new dataset, richer metrics, prompt sensitivity analysis, prompt ensembling, and reproducibility tooling.

The final verified run uses the stronger `630k-audioset-best.pt` checkpoint rather than the exact original checkpoint configuration associated with the paper comparison. Because of that, the work should be described honestly as:

- a **full benchmark reproduction with an improved checkpoint**;
- plus **extension experiments and analysis**.

2 Background and Task Formulation

2.1 Main Task

The central task in this project is **zero-shot audio classification**. Given an audio clip and a set of class labels, the model must predict the correct class *without* task-specific supervised training on that dataset.

Instead of learning a dataset-specific classifier head, we convert class labels into natural-language prompts and compare the audio embedding against text embeddings. Conceptually, this means that prediction is reduced to a retrieval problem in a shared embedding space.

2.2 Paper Idea and CLAP Intuition

CLAP learns two encoders jointly:

- an audio encoder $f_a(\cdot)$;
- a text encoder $f_t(\cdot)$,

where $X_a \in \mathbb{R}^{F \times T}$ is the processed audio (F spectral components, T time bins) and X_t is the text input. For a batch of N audio-text pairs $\{X_a, X_t\}_i$, the encoders produce intermediate representations:

$$\hat{X}_a = f_a(X_a), \qquad \hat{X}_t = f_t(X_t), \tag{1}$$

where $\hat{X}_a \in \mathbb{R}^{N \times V}$ and $\hat{X}_t \in \mathbb{R}^{N \times U}$. These are then brought into a joint multimodal space of dimension d via learnable linear projections L_a and L_t:

$$E_a = L_a(\hat{X}_a), \qquad E_t = L_t(\hat{X}_t), \tag{2}$$

where $E_a, E_t \in \mathbb{R}^{N\times d}$. The similarity matrix over the batch is computed as:

$$C = \tau \cdot (E_t \cdot E_a^\top), \tag{3}$$

where τ is a learnable temperature parameter. $C \in \mathbb{R}^{N\times N}$ has N correct pairs on the diagonal and $N^2 - N$ incorrect pairs off-diagonal. The model is trained with the symmetric cross-entropy loss:

$$\mathcal{L} = 0.5 \cdot \big(\ell_{\text{text}}(C) + \ell_{\text{audio}}(C)\big). \tag{4}$$

For zero-shot classification, given a class set $\mathcal{C} = \{c_1, \ldots, c_K\}$, we build prompts $p(c_1), \ldots, p(c_K)$, compute their projected text embeddings $E_t^{(k)}$, and predict using cosine similarity:

$$s(X_a,\, p(c_k)) = \cos\!\big(E_a,\, E_t^{(k)}\big) = \frac{E_a \cdot E_t^{(k)\top}}{\|E_a\|\,\|E_t^{(k)}\|}, \tag{5}$$

$$\hat{y} = \arg\max_k \; s\big(X_a,\, p(c_k)\big). \tag{6}$$

2.3 Prompting and Prompt Ensembling

Prompt phrasing matters. A class such as `dog barking` can be written as:

- "this is a sound of dog barking",
- "the audio contains dog barking",
- "a recording of dog barking",

and each phrasing yields a slightly different text embedding.

This project therefore studies not only single-prompt inference but also prompt ensembling. Let $\mathcal{T} = \{t_1, \ldots, t_M\}$ be the prompt family and let $p_{k,m} = t_m(c_k)$ be the m-th prompt template for class c_k. The audio and projected text embeddings for sample i are:

$$E_a^{(i)} = L_a\big(f_a(X_a^{(i)})\big), \qquad Q_{k,m} = L_t\big(f_t(p_{k,m})\big). \tag{7}$$

For a single prompt template, the class score is:

$$s_k^{(m)} = \cos\!\big(E_a^{(i)},\, Q_{k,m}\big). \tag{8}$$

Uniform prompt ensembling averages the projected text embeddings:

$$Q_k^{\text{ens}} = \frac{1}{M}\sum_{m=1}^{M} Q_{k,m}. \tag{9}$$

Weighted prompt ensembling uses:

$$Q_k^{\text{ens}} = \sum_{m=1}^{M} w_m\, Q_{k,m}, \qquad \sum_{m=1}^{M} w_m = 1. \tag{10}$$

The final prediction is then:

$$\hat{y}_i = \arg\max_k \cos\big(E_a^{(i)},\, Q_k^{\text{ens}}\big). \tag{11}$$

We implemented both uniform and entropy-weighted ensembling. One of the most important negative findings of the project is that this strategy does not produce reliable gains.

3 Method

3.1 System Overview

The project uses two repositories together:

- `clap-official`: the official LAION CLAP implementation, which provides the actual `laion_clap` model code;
- `clap-reimpl`: our experiment layer, which provides dataset loading, prompt generation, metric computation, run wrappers, figures, manifests, and the final report.

The evaluation pipeline follows the same high-level procedure across datasets:

1. prepare audio paths, labels, and metadata for the dataset;
2. generate text prompts from a prompt family suited to the domain;
3. compute CLAP audio embeddings and CLAP text embeddings;
4. compute similarity scores and obtain ranked predictions;
5. aggregate results into metrics, figures, JSON reports, and summary tables.

The design intentionally separates:

- model implementation,
- dataset preparation,
- benchmark logic,
- reporting and figures,
- reproducibility logging.

This separation mattered practically because it allowed us to stabilize the environment, change checkpoints, and add new datasets without rewriting the core model code.

3.2 Dataset Abstractions and Prompt Families

The main benchmark runner is `scripts/analysis/run_zeroshot_metrics.py`. Its design centers on a `DatasetBundle` abstraction containing:

- dataset key,

- display name,
- root directory,
- prompt family,
- class names,
- audio files,
- numeric labels,
- dataset-specific metadata.

Supported datasets are:

- `esc50`,
- `urbansound8k`,
- `gtzan`,
- `fsdd`.

Three prompt families are used:

- `sound`,
- `music`,
- `speech`.

Datasets are assigned families as follows:

- `ESC-50` and `UrbanSound8K`: `sound`;
- `GTZAN`: `music`;
- `FSDD`: `speech`.

This matters because prompt phrasing produces visibly different results on some datasets, especially `GTZAN` and `FSDD`.

3.3 Caching and Generated Outputs

Audio and text embeddings are cached under `outputs/embeddings/<dataset>/`. The cache contains:

- `audio_embeddings.npy`,
- `labels.npy`,
- `audio_files.json`,
- prompt-specific text embedding arrays,
- ensemble embeddings.

Caching made repeated evaluation practical and significantly reduced rerun time during debugging and extension experiments.

For each dataset, the pipeline generates:

- metrics JSON,
- confusion matrix,
- reliability diagram,
- per-class accuracy plot,
- prompt sensitivity plot,
- ensemble comparison plot.

At the global level, it also generates:

- summary CSV tables,
- summary plots,
- run manifests,
- acceptance reports.

3.4 Environment and Engineering Notes

The final verified run used:

- WSL2 with Ubuntu 22.04,
- Python 3.10,
- `torch 2.4.1+cu121`,
- `numpy 1.26.4`,
- `transformers 4.30.0`,
- checkpoint `630k-audioset-best.pt`.

One implementation detail mattered for stability: a small NumPy compatibility shim was added so that the official CLAP data path worked correctly in the final environment. This should be treated as part of the method rather than as an irrelevant implementation accident.

3.5 Datasets and Evaluation Scope

Dataset	Samples	Classes	Domain	Track	Default Prompt Family
`ESC-50`	2000	50	Environmental sound	Baseline + Extension	sound
`UrbanSound8K`	8732	10	Urban sound	Baseline + Extension	sound
`GTZAN`	999	10	Music genre	Baseline + Extension	music
`FSDD`	3000	10	Spoken digits	Extension only	speech

Table 1: Datasets used in the final verified study.

Two experiment tracks were run:

- **Baseline track:** `ESC-50`, `UrbanSound8K`, and `GTZAN`;
- **Extension track:** `ESC-50`, `UrbanSound8K`, `GTZAN`, and `FSDD`.

The baseline track reproduces the original benchmark-style dataset suite in the verified setup. The extension track evaluates the improved checkpoint, richer metrics, prompt sensitivity, and prompt ensembling.

3.6 Reproducibility and Dataset Preparation

The final artifact set includes:

- dataset verification,
- checkpoint manifest generation,
- baseline and extension manifests,
- GTZAN preparation manifest,
- final acceptance check.

One special preparation step was required for `GTZAN`. The raw archive was converted from `.au` to `.wav`, then normalized to the expected benchmark form by removing the known problematic sample `jazz.00054.wav`. The final acceptance report passed with `all_ok = true`.

4 Evaluation Setup and Metrics

4.1 Metrics

This project extends the original reporting style by adding several metrics beyond top-1 accuracy.

Top-1 Accuracy

$$\text{Accuracy} = \frac{1}{N} \sum_i \mathbf{1}[\hat{y}_i = y_i]. \tag{12}$$

Macro-F1

For each class k, let precision be P_k and recall be R_k:

$$F1_k = \frac{2P_k R_k}{P_k + R_k}, \qquad \text{Macro-F1} = \frac{1}{K} \sum_k F1_k. \tag{13}$$

Macro-F1 treats all classes equally and is useful when performance is uneven across classes.

Balanced Accuracy

$$\text{Balanced Accuracy} = \frac{1}{K} \sum_k \text{Recall}_k. \tag{14}$$

Top-5 Accuracy

$$\text{Top-5} = \frac{1}{N} \sum_i \mathbf{1}[y_i \in \text{top-5 ranked classes for sample } i]. \tag{15}$$

Mean Reciprocal Rank (MRR)

If rank_i is the rank of the correct class for sample i, then:

$$\text{MRR} = \frac{1}{N} \sum_i \frac{1}{\text{rank}_i}. \tag{16}$$

Expected Calibration Error (ECE)

$$\text{ECE} = \sum_b \frac{|B_b|}{N} \left| \text{acc}(B_b) - \text{conf}(B_b) \right|, \tag{17}$$

where B_b is the set of predictions in confidence bin b.

Prompt Sensitivity Score

Prompt sensitivity captures the spread between best and worst prompt performance:

$$\text{PSS} = \text{best prompt score} - \text{worst prompt score}. \tag{18}$$

4.2 What the Metric Suite Adds

The richer metric set is important because it reveals behavior that plain accuracy would hide:

- `GTZAN` has moderate top-1 accuracy but very strong Top-5 behavior, meaning the model often ranks the correct genre near the top.
- `FSDD` has poor top-1 accuracy but noticeably higher Top-5, suggesting weak partial awareness rather than complete collapse.
- `UrbanSound8K` behaves better under balanced accuracy than a single top-1 number might suggest.

5 Results

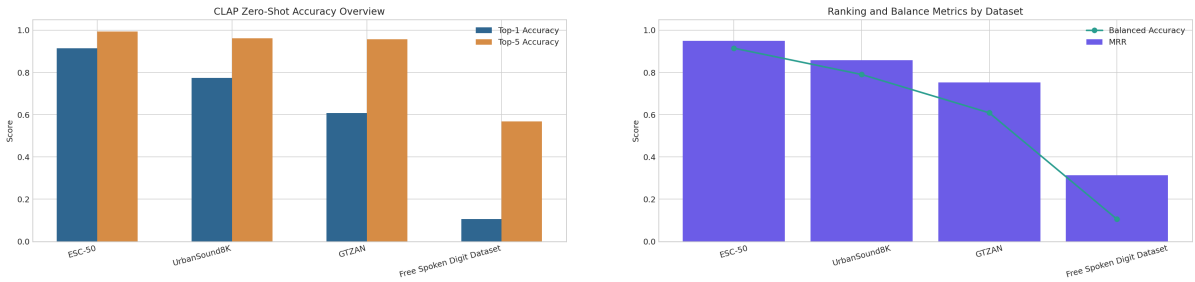
5.1 Main Benchmark Results

Dataset	Accuracy	Macro-F1	Bal. Acc.	Top-5	MRR	ECE
`ESC-50`	0.9150	0.9120	0.9150	0.9935	0.9499	0.8833
`UrbanSound8K`	0.7747	0.7786	0.7905	0.9612	0.8582	0.6382
`GTZAN`	0.6086	0.5878	0.6089	0.9570	0.7529	0.4857
`FSDD`	0.1053	0.0629	0.1053	0.5677	0.3133	0.0003

Table 2: Final zero-shot performance on the four evaluated datasets.

The results show a clear hierarchy of difficulty:

- `ESC-50` is the strongest result and confirms that CLAP transfers well to environmental sound events.
- `UrbanSound8K` remains strong, though prompt wording affects it more noticeably.
- `GTZAN` is moderate rather than dominant, with confusion among semantically close genres.
- `FSDD` is the clearest failure case and shows that general audio-language pretraining is not sufficient for fine-grained spoken-digit recognition in zero-shot mode.



(a) Accuracy overview across datasets.

(b) Ranking and balance behavior.

Figure 1: Summary plots generated from the final evaluation pipeline.

5.2 Improvements Beyond the Original Benchmark

This project includes three main improvements beyond a plain rerun:

1. addition of `FSDD`;
2. addition of richer metrics;
3. addition of prompt ensembling analysis.

Improvement 1: FSDD

`FSDD` was added as a speech-focused diagnostic dataset. Its purpose is to test whether a general audio-language representation transfers to a fine-grained speech classification task.

Result:

- top-1 accuracy: `0.1053`

Interpretation:

- performance is close to random for a 10-class task;
- this is strong evidence that CLAP is weak on zero-shot spoken-digit recognition.

Improvement 2: Richer Metrics

The richer evaluation makes several important things visible that plain accuracy would hide:

- `GTZAN` is much stronger as a ranking model than as a hard classifier.
- `UrbanSound8K` has better balanced-accuracy behavior than its single accuracy number suggests.
- `FSDD` has weak top-1 transfer but nontrivial ranking structure.

Improvement 3: Prompt Ensembling

Prompt ensembling was inspired by CLIP-style prompt ensembling in image-text models. We implemented:

- uniform averaging;
- entropy-weighted averaging.

Dataset	Worst Prompt	Best Prompt	Uniform Ensemble	Δ vs Best
`ESC-50`	0.9150	0.9265	0.9270	+0.0005
`UrbanSound8K`	0.7747	0.8106	0.7989	-0.0117
`GTZAN`	0.5966	0.6647	0.6537	-0.0110
`FSDD`	0.0740	0.1057	0.0757	-0.0300

Table 3: Prompt ensembling results.

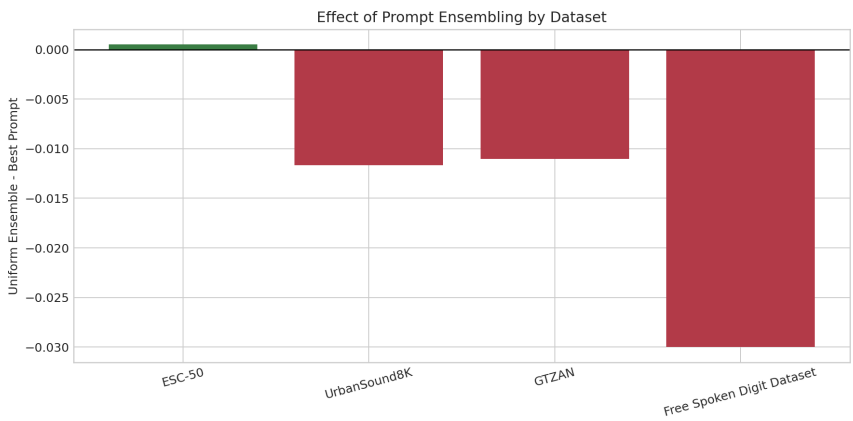
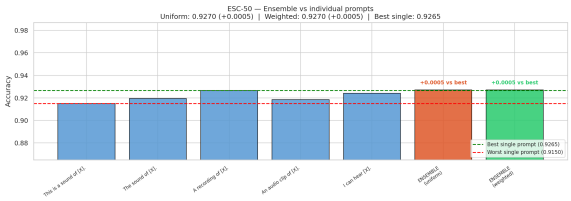


Figure 2: Effect of prompt ensembling relative to the best single prompt.

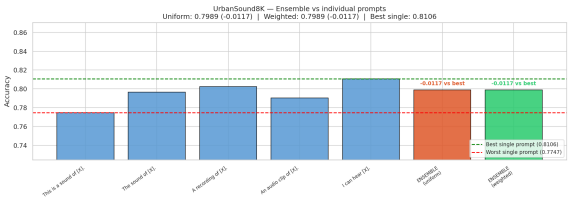
Prompt ensembling is the strongest negative result in the project. It gives only a tiny gain on `ESC-50` and reduces performance on `UrbanSound8K`, `GTZAN`, and `FSDD`. The entropy-weighted ensemble did not improve over the uniform ensemble in the final runs because the weights effectively collapsed to near-uniform values. In practice, CLAP did not separate the prompt variants strongly enough for the weighting strategy to matter.

5.3 Visual Diagnostics

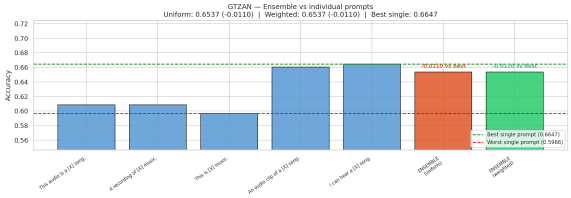
Ensemble Comparison Figures



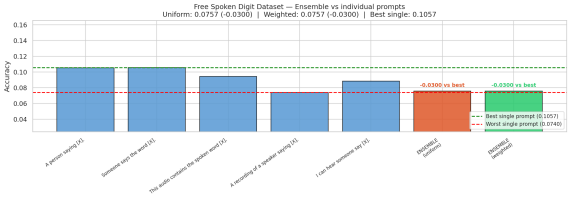
(a) `ESC-50`.



(b) `UrbanSound8K`.



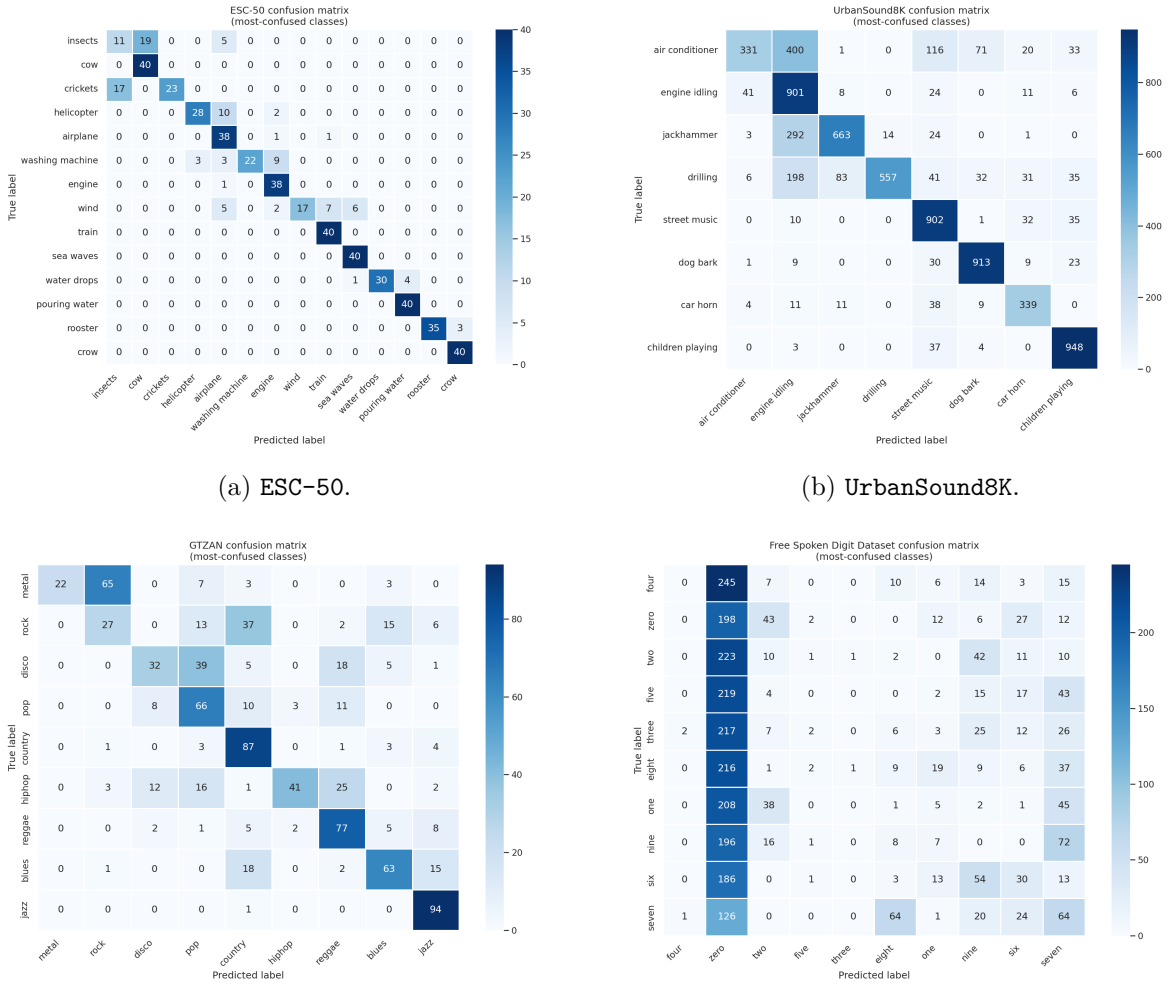
(c) `GTZAN`.



(d) `FSDD`.

Figure 3: Ensemble comparison plots for all datasets.

Confusion Matrices



(a) ESC-50.

(b) UrbanSound8K.

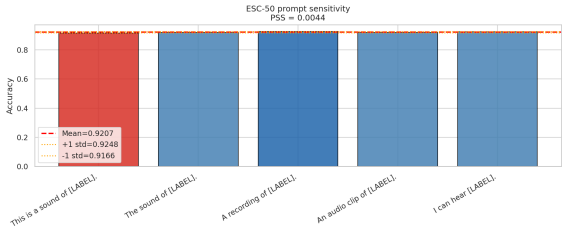
(c) GTZAN.

(d) FSDD.

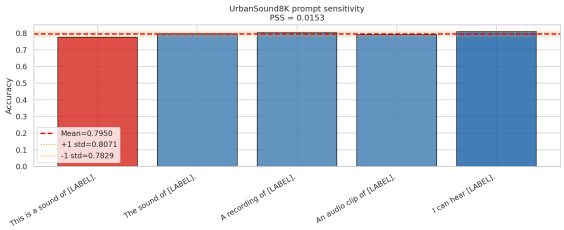
Figure 4: Confusion matrices for all four datasets.

The confusion matrices reinforce the metric-level story. GTZAN errors cluster among related genres, while FSDD shows broad confusion across digit classes and no strong zero-shot structure.

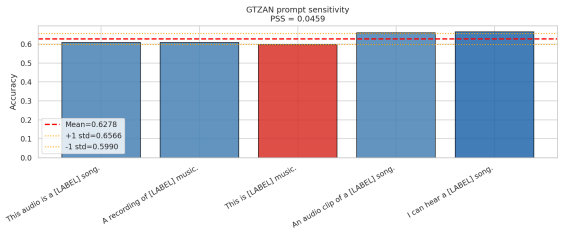
Prompt Sensitivity Plots



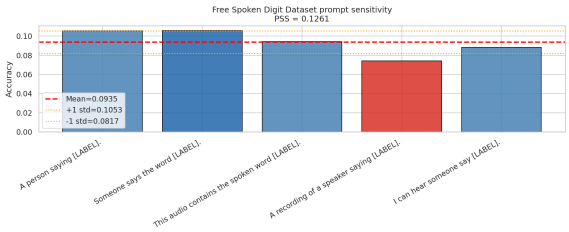
(a) ESC-50.



(b) UrbanSound8K.



(c) GTZAN.

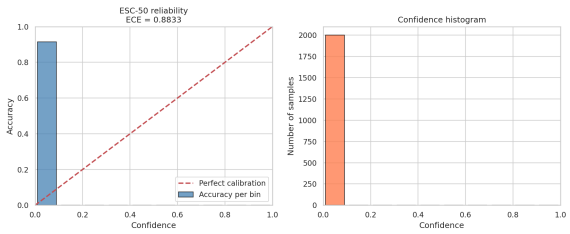


(d) FSDD.

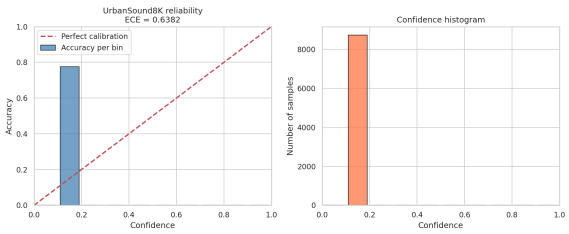
Figure 5: Prompt sensitivity across all datasets.

These plots show that prompt wording matters more on some datasets than others, which helps explain why ensembling can fail: averaging can dilute the signal of the best prompt.

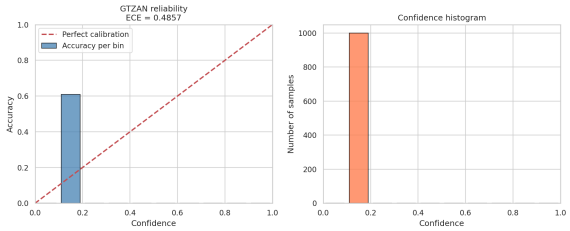
Reliability Diagrams



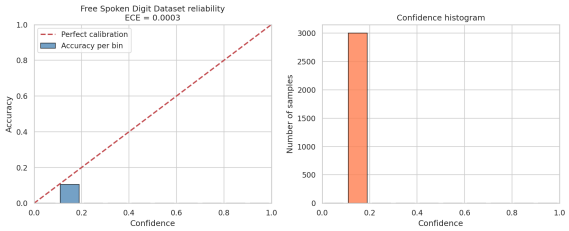
(a) ESC-50.



(b) UrbanSound8K.



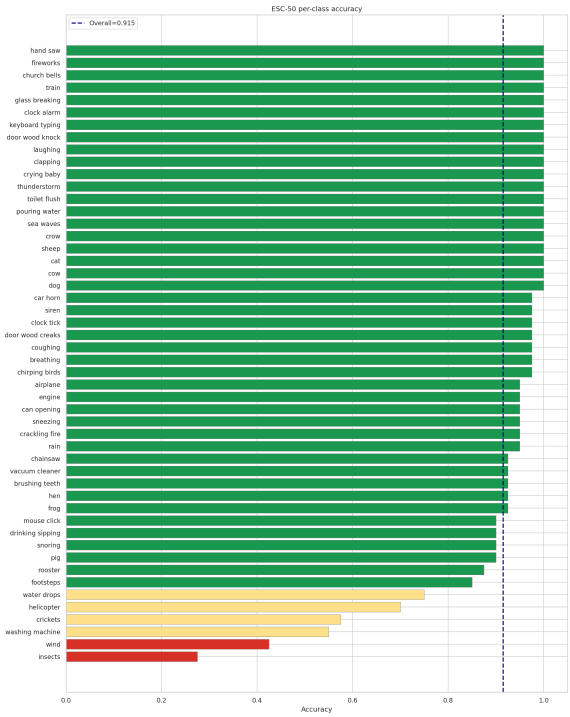
(c) GTZAN.



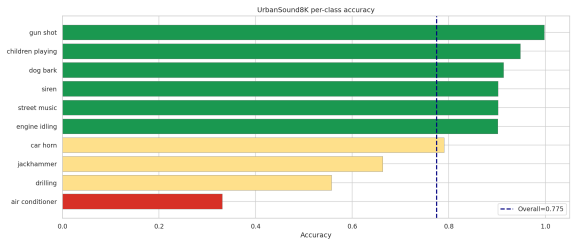
(d) FSDD.

Figure 6: Reliability diagrams for all datasets.

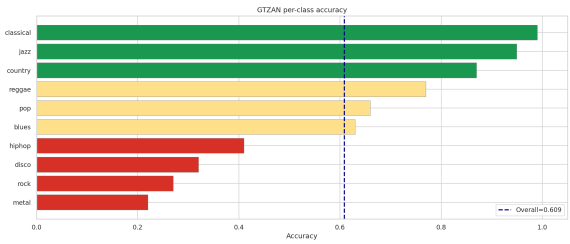
Per-Class Accuracy



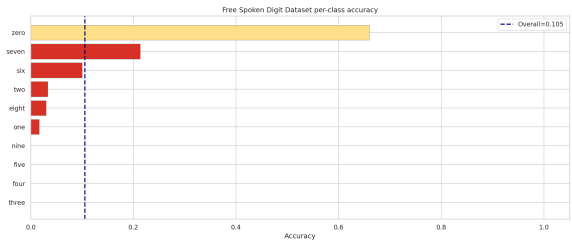
(a) `ESC-50`.



(b) `UrbanSound8K`.



(c) `GTZAN`.



(d) `FSDD`.

Figure 7: Per-class accuracy plots for all datasets.

6 Discussion

The project produces a balanced scientific picture of CLAP:

- it is genuinely strong on environmental sound recognition;
- it remains useful on urban sound recognition;
- it is workable but limited on music genre classification;
- it is weak on fine-grained spoken-digit recognition.

The most important insight is that CLAP's transfer ability is **domain-sensitive**, not uniformly strong.

Three points are especially important in the final write-up:

- the benchmark reproduction was successful, but it uses a stronger checkpoint than the one emphasized in the original comparison;
- richer metrics provide a more honest description than top-1 accuracy alone;
- prompt ensembling should be reported as a validated negative result rather than as a successful improvement.

6.1 What Should Be Claimed in the Final Submission

The strongest claims supported by the final run are:

- The benchmark suite `ESC-50`, `UrbanSound8K`, and `GTZAN` was reproduced in a verified WSL pipeline.
- The final benchmark uses the stronger `630k-audioset-best.pt` checkpoint, so it should be described as an improved-checkpoint reproduction rather than a strict same-checkpoint replication.
- `FSDD` was added as a new diagnostic dataset.
- Multiple richer metrics were added beyond plain accuracy.
- Prompt ensembling was implemented and evaluated.
- Prompt ensembling does not reliably improve CLAP.
- Entropy-weighted prompt ensembling collapses to effectively uniform behavior.
- CLAP transfers poorly to zero-shot spoken-digit recognition on `FSDD`.

6.2 Limitations

- The final benchmark comparison is not a strict same-checkpoint replication of the original paper.
- The environment had to be stabilized to a working CUDA-compatible WSL stack, so the exact package mix should be treated as part of the reported method.
- `GTZAN` required explicit raw-data conversion from `.au` to `.wav` and removal of the known corrupted sample.
- The extension conclusions are strong empirically, but they are still bounded by the prompt families and checkpoint used in this study.

7 Conclusion

This project delivered:

- a verified CLAP benchmark run over four datasets,
- a reproducible WSL execution setup,
- richer evaluation metrics,

- a new diagnostic dataset,
- a clear extension study on prompt ensembling.

The main final message is balanced rather than purely celebratory: CLAP is genuinely strong for some zero-shot audio tasks, but its gains are uneven across domains, and prompt ensembling is not a dependable shortcut to better performance. That combination of successful reproduction, practical engineering, and validated negative-result analysis makes this project stronger than a plain rerun of the original benchmark.